

Práctica en clase. Uso de Client URL (cURL).

Instrucciones. Práctica utilizando servicios para la comunicación en red en un equipo Linux sobre VirtualBox.

1. Realiza la práctica que viene en la siguiente página.
2. Crear un **archivo de Word** con una portada con tu nombre. Guárdelo con el nombre **NombreAlumno_cURL.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 1. Coloca la captura de pantalla del resultado de la ejecución del comando **curl** utilizando el método **OPTIONS** hacia la página de la UV.
 2. Coloca la captura de pantalla del resultado de la ejecución del comando **curl** utilizando el método **GET** hacia la página de la UV.
 3. Coloca la captura de pantalla del resultado de la ejecución del comando **curl** utilizando el método **POST** para insertar datos hacia el API de ejemplo *JsonPlaceHolder*.
 4. Coloca la captura de pantalla del resultado de la ejecución del comando **curl** utilizando el método **PUT** para modificar datos hacia el API de ejemplo *JsonPlaceHolder*.
 5. Coloca la captura de pantalla del resultado de la ejecución del comando **curl** utilizando el método **PATCH** para modificar datos hacia el API de ejemplo *JsonPlaceHolder*.
 6. Coloca la captura de pantalla del resultado de la ejecución del comando **curl** utilizando el método **DELETE** para eliminar datos hacia el API de ejemplo *JsonPlaceHolder*.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**. *Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).*
2. Esta práctica tiene como prerrequisito contar con un equipo de cómputo con suficientes recursos (Espacio en disco duro, RAM y CPU) para soportar la instalación del software y la ejecución de las máquinas virtuales.

Práctica. Uso de Client URL en los métodos HTTP más comunes.

El **comando cURL** (Client URL o cURL) es una línea de comandos que permite el intercambio de datos entre un dispositivo y un servidor a través de un terminal. Con esta interfaz de línea de comandos (o CLI), puede especificarse la URL de un servidor (es decir, la ubicación a la que se envía la solicitud) y los datos que se van a enviar a ese servidor.

Algunas herramientas de API, como Postman e Insomnia, tienen una interfaz de usuario (IU) interactiva que permite hacer diferentes tipos de solicitudes a URL, tanto para recibirlas como para procesarlas. El comando cURL hace lo mismo, pero lo hace directamente en el terminal, y funciona en Linux, Mac y Windows.

En esta práctica usaremos la API ficticia [JSONPlaceholder](#) para explicar las distintas formas en las que puede usarse este comando. Esta API ficticia contiene diferentes rutas de ejemplo para realizar solicitudes.

Instrucciones.

1. Ingresa a una terminal de tu equipo Linux.
2. Para comprobar la versión de cURL instalada utilizando este comando:

```
$ curl --version
curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9
```

3. Para ver la lista de opciones puede ejecutar:

```
$ curl --help
-d, --data <data>          Datos HTTP POST
-f, --fail                 Error silencioso (sin salida en absoluto) en errores HTTP
-h, --help <category>     Obtener ayuda para los comandos
-i, --include              Incluir encabezados de respuesta de protocolo en la salida
-o, --output <file>       Escribir en un archivo en lugar de stdout
-O, --remote-name          Escribir la salida en un archivo denominado archivo remoto
-s, --silent              Modo silencioso
-T, --upload-file <file>  Transferir el ARCHIVO local al destino
-u, --user <user:password> Usuario y contraseña del servidor
-A, --user-agent <name>   Enviar User-Agent <nombre> al servidor
-v, --verbose              Modo verboso
-V, --version              Mostrar el número de versión y salir
```

Método OPTIONS

1. Comencemos probando el método **OPTIONS** que es utilizado para describir las opciones de comunicación para el recurso de destino. Código de estado devuelto: Tanto 200 Aceptar como 204 Sin contenido son códigos de estado permitidos.
2. Escriba la petición de la siguiente forma.

```
$ curl -X OPTIONS https://www.uv.mx/ -i
HTTP/2 200
allow: OPTIONS, TRACE, GET, HEAD, POST
server: Microsoft-IIS/10.0
```

```
public: OPTIONS, TRACE, GET, HEAD, POST
x-powered-by: ASP.NET
date: Thu, 15 Feb 2024 04:42:17 GMT
content-length: 0
```

3. Observe la respuesta del servidor. Utiliza **HTTP versión 2**, permite los métodos **OPTIONS**, **TRACE**, **GET**, **HEAD** y **POST**. Es un servidor web **IIS versión 10** que soporta **ASP.NET**.
4. Realice lo mismo contra el servidor de Google.

```
pato@equipoA:~$ curl -X OPTIONS https://www.google.com -i
```

```
pato@equipoA:~$ curl -X OPTIONS https://www.google.com -i
HTTP/2 405
allow: GET, HEAD
date: Thu, 15 Feb 2024 04:45:26 GMT
content-type: text/html; charset=UTF-8
server: gws
content-length: 1592
x-xss-protection: 0
x-frame-options: SAMEORIGIN
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000

<!DOCTYPE html>
<html lang=en>
  <meta charset=utf-8>
  <meta name=viewport content="initial-scale=1, minimum-scale=1, width=device-width">
  <title>Error 405 (Method Not Allowed)!!1</title>
  <style>
    *{margin:0;padding:0}html,code{font:15px/22px arial,sans-serif}html{background:#fff;color:#222;padding:15px}body{margin:7% auto 0;max-width:390px;min-height:180px;padding:30px 0 15px}* > body{background:url(//www.google.com/images/errors/robot.png) 100% 5px no-repeat;padding-right:205px}p{margin:0 11px 0 22px;overflow:hidden}ins{color:#777;text-decoration:none}a img{border:0}@media screen and (max-width:772px){body{background:none;margin-top:0;max-width:none;padding-right:0}}#logo{background:url(//www.google.com/images/branding/googlelogo/1x/googlelogo_color_150x54dp.png) no-repeat;margin-left
```

5. Observe como además del encabezado de la respuesta, se obtiene el cuerpo de la misma indicando que **OPTIONS** no es un método que permita Google a sus servidor. Para evitar toda esta información adicional, agregue **--head** a su consulta.

```
pato@equipoA:~$ curl -X OPTIONS https://www.google.com -i --head
HTTP/2 405
allow: GET, HEAD
date: Thu, 15 Feb 2024 04:47:43 GMT
content-type: text/html; charset=UTF-8
server: gws
content-length: 1592
x-xss-protection: 0
x-frame-options: SAMEORIGIN
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```

6. Aunque nos sigue devolviendo un código **405 (Método no permitido)**, nos devuelve el encabezado que nos da información sobre los métodos permitidos, pero no sobre los detalles del servidor web. Esta es la manera más común de llamar al método **OPTIONS**, utilizando **--head**.
7. Como en **www.uv.mx** existe una redirección desde **HTTP** hacia **HTTPS**. Ahora intente solicitar un **OPTIONS** usando **HTTP** no seguro.

```
pato@equipoA:~$ curl -X OPTIONS http://www.uv.mx -i
HTTP/1.1 301 Moved Permanently
Content-Type: text/html; charset=UTF-8
Location: https://www.uv.mx/
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Thu, 15 Feb 2024 04:51:24 GMT
Content-Length: 141
<head><title>Document Moved</title></head>
<body><h1>Object Moved</h1>This document may be found <a HREF="https://www.uv.mx/">here</a></body>
```

8. Observe como el encabezado nos envía un código **301 (Redirección permanente)** que nos indica que el servidor realiza una redirección en este recurso. cURL no navega hacia esa redirección sino se le indica. Para eso, debe agregar la opción **-L** que es la abreviación de **--location**.

```
pato@equipoA:~$ curl -X OPTIONS http://www.uv.mx -iL
```

```
pato@equipoA:~$ curl -X OPTIONS http://www.uv.mx -iL
HTTP/1.1 301 Moved Permanently
Content-Type: text/html; charset=UTF-8
Location: https://www.uv.mx/
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Thu, 15 Feb 2024 04:54:52 GMT
Content-Length: 141

HTTP/2 200
allow: OPTIONS, TRACE, GET, HEAD, POST
server: Microsoft-IIS/10.0
public: OPTIONS, TRACE, GET, HEAD, POST
x-powered-by: ASP.NET
date: Thu, 15 Feb 2024 04:54:53 GMT
content-length: 0
```

9. Observe como primero va a **HTTP** y luego brinca a la redirección hacia **HTTPS**.
10. Intente hacerlo con Google. Como este no permite el método **OPTIONS**, agregue **--head** a la petición.

```
pato@equipoA:~$ curl -X OPTIONS https://www.google.com -iL --head
HTTP/2 405
allow: GET, HEAD
date: Thu, 15 Feb 2024 04:47:43 GMT
content-type: text/html; charset=UTF-8
server: gws
content-length: 1592
x-xss-protection: 0
x-frame-options: SAMEORIGIN
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```

11. Observe como aquí **cURL** no redirecciona pues Google no permite este método.
12. Pruebe con la API de la práctica.

```
pato@equipoA:~$ curl -X OPTIONS https://jsonplaceholder.typicode.com -iL --head
HTTP/2 204
date: Thu, 15 Feb 2024 05:09:47 GMT
```

```
report-to: {"group":"heroku-nel","max_age":3600,"endpoints":[{"url":"https://nel.heroku.com/reports?ts=1707973787&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&s=Y3m6uSt4en3acKKngS6HlcLt%2FBKePJHOI%2BQ465MUjEs%3D"}]}
reporting-endpoints: heroku-nel=https://nel.heroku.com/reports?ts=1707973787&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&s=Y3m6uSt4en3acKKngS6HlcLt%2FBKePJHOI%2BQ465MUjEs%3D
nel: {"report_to":"heroku-nel","max_age":3600,"success_fraction":0.005,"failure_fraction":0.05,"response_headers":["Via"]}
x-powered-by: Express
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
x-ratelimit-reset: 1707973837
vary: Origin, Access-Control-Request-Headers
access-control-allow-credentials: true
access-control-allow-methods: GET,HEAD,PUT,PATCH,POST,DELETE
via: 1.1 vegur
cf-cache-status: DYNAMIC
server: cloudflare
cf-ray: 855b0a6d7b53e583-DFW
alt-svc: h3=":443"; ma=86400
```

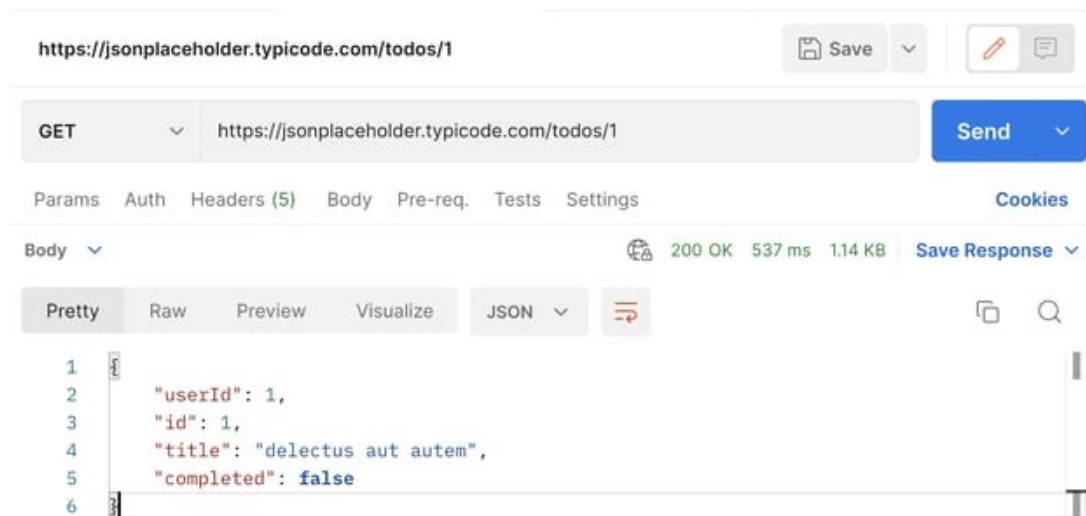
13. Observe como aquí si funciona y podemos ver los detalles del servidor en la respuesta. Este servidor también funciona sin **HTTP** y devuelve el mismo resultado.

Método GET

1. El método **GET** solicita una representación de un recurso específico. Las peticiones que usan el método **GET** sólo deben recuperar datos. Generalmente regresan el código **200**.

```
pato@equipoA:~$ curl -X GET https://jsonplaceholder.typicode.com/todos/1
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

2. Este resultado es similar al que obtendríamos de plataformas como **Postman**.



3. Puede agregar la opción `-i` para ver el código de estado devuelto.

```
pato@equipoA:~$ curl -X GET https://jsonplaceholder.typicode.com/todos/1 -i
HTTP/2 200
date: Thu, 15 Feb 2024 06:12:05 GMT
...
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

Método POST

1. El método `POST` envía datos al servidor. El tipo del cuerpo de la solicitud es indicado por la cabecera `Content-Type`. Para hacer una solicitud `POST` a una URL, usa la opción `-X` y se pasa el método `POST` como el valor. Para agregar datos con la solicitud, se usa la opción `-d` (guion y d minúscula), que es un alias de `--data`. Al enviar datos con una solicitud, se puede usar dos formatos muy comunes: `application/x-www-form-urlencoded` o `application/json`.
2. Si se especifica ningún formato, el comando cURL usa `application/x-www-form-urlencoded` de forma predeterminada. Este ejemplo usa este formato y la API ficticia JSON. Observe como los datos se deben de ir en formato de clave-valor separados por un ampersand. Se utiliza la opción `-H` (guion y H mayúscula), que es un alias de `--header`. La respuesta esperada es código `201`.

```
pato@equipoA:~$ curl -X POST -d "name=Ejemplo Curl&type=articulo" -H "Content-Type: application/x-www-form-urlencoded" https://jsonplaceholder.typicode.com/posts
{
  "name": "Ejemplo Curl",
  "type": "articulo",
  "id": 101
}
```

3. El API responde con los datos del registro agregado.
4. Con cURL, también se puede enviar un objeto JSON en forma de cadena. Solo hay que especificar el `Content-Type` como `application/json`. Se utiliza la opción `-H` (guion y H mayúscula), que es un alias de `--header`.

```
pato@equipoA:~$ curl -X POST -d '{"name": "Ejemplo JSON", "type": "articulo"}' -H "Content-Type: application/json" https://jsonplaceholder.typicode.com/posts
{
  "name": "Ejemplo JSON",
  "type": "articulo",
  "id": 101
}
```

5. Puede agregar la opción `-i` para ver el código de estado devuelto.

```
pato@equipoA:~$ curl -X POST -d '{"name": "Ejemplo JSON", "type": "articulo"}' -H "Content-Type: application/json" https://jsonplaceholder.typicode.com/posts -i
HTTP/2 201
...
{
```

```
"name": "Ejemplo JSON",  
"type": "articulo",  
"id": 101  
}
```

- Si quiere utilizar un archivo con los datos en JSON, se puede enviar junto con la petición. Cree un nuevo archivo en este mismo directorio con `$nano data.json`. Coloque el siguiente contenido.

```
{"name": "Ejemplo JSON", "type": "articulo"}
```

```
GNU nano 6.2 data.json  
{"name": "Ejemplo JSON", "type": "articulo"}
```

- Guarde el archivo con `Ctrl + O` y Enter. Para salir `Ctrl + X`. Para referirse a su archivo de datos, utilice el carácter de arroba `@` y la ruta del archivo.

```
pato@equipoA:~$ curl -X POST -d @data.json -H "Content-Type: application/json"  
https://jsonplaceholder.typicode.com/posts  
{  
  "name": "Ejemplo JSON",  
  "type": "articulo",  
  "id": 101  
}
```

Método PUT

- Con el método `PUT` y el comando `cURL`, se puede hacer solicitudes "de actualización" a una API para modificar un recurso existente. En la API ficticia JSON, se especifican la ruta del recurso y el método `PUT`, y se pasan los datos para actualizar el recurso. Normalmente regresa código `200`.
- Estos datos pueden tener el formato que sea. En este ejemplo, usaremos `application/json`.

```
pato@equipoA:~$ curl -X PUT -d '{"name": "Prueba de put", "type": "ejemplo"}' -H "Content-Type:  
application/json" https://jsonplaceholder.typicode.com/posts/1  
{  
  "name": "Prueba de put",  
  "type": "ejemplo",  
  "id": 1  
}
```

Método DELETE

- También se puede enviar solicitudes de eliminación a una API mediante el método `DELETE` y el comando `cURL`, y la URL y los datos que proporcionen en esta solicitud dependerán de la configuración de la API. En la API ficticia JSON, la ruta del recurso y el método `DELETE` se especifican de esta forma.

```
pato@equipoA:~$ curl -X DELETE https://jsonplaceholder.typicode.com/posts/1
```

```
{}
```

2. **DELETE** también devuelve código **200** cuando tiene éxito.

Método PATCH

1. El método **PATCH** aplica modificaciones parciales a un recurso. Normalmente regresa código **200**. Estos datos pueden tener el formato que sea. En este ejemplo, usaremos **application/json**.

```
pato@equipoA:~$ curl -X PATCH -d '{"title": "Titulo de ejemplo", "body": "Cuerpo de ejemplo", "name": "Prueba de put", "type": "ejemplo"}' -H "Content-Type: application/json" https://jsonplaceholder.typicode.com/posts/1
{
  "userId": 1,
  "id": 1,
  "title": "Titulo de ejemplo",
  "body": "Cuerpo de ejemplo",
  "name": "Prueba de put",
  "type": "ejemplo"
}
```

3. Observe como la API devuelve todos los datos del registro para ejemplificar que se pueden modificar solo algunos campos.

Uso de autenticación

1. Se puede también entrar a un recurso protegido con algún mecanismo de autenticación. Vamos a hacer uso de la autenticación **JWT** que se utiliza principalmente en servicios web.
2. Vamos a autenticarnos hacia Eminus. Para no replicar el ofuscamiento de la cadena de password en Eminus, utilicemos el JWT que obtenemos desde el navegador. Así también evitamos escribir la contraseña en texto plano en cURL.
3. Primeramente, intente hacer una petición hacia Eminus para obtener los datos de un usuario. La URL de ese **endpoint** es **https://eminusapi.uv.mx/eminusapi/api/Usuario/getUsuario**.

```
pato@equipoA:~$ curl -X GET https://eminusapi.uv.mx/eminusapi/api/Usuario/getUsuario
```

```
pato@equipoA:~$ curl -X GET https://eminusapi.uv.mx/eminusapi/api/Usuario/getUsuario
pato@equipoA:~$
```

4. Observe que el resultado es vacío. Eminus no sabe quien esta haciendo la petición por lo que simplemente no envía un cuerpo de respuesta. Agregue la opción **-i** para ver el encabezado.

```
pato@equipoA:~$ curl -X GET https://eminusapi.uv.mx/eminusapi/api/Usuario/getUsuario -i
HTTP/2 401
server: Kestrel
www-authenticate: Bearer
x-powered-by: ASP.NET
date: Thu, 15 Feb 2024 15:04:16 GMT
content-length: 0
```

5. Observe como nos esta indicando que para poder hablar con este **endpoint**, requerimos enviar nuestras credenciales utilizando la autenticación **Bearer**. Eso quiere decir, envíale una cadena token.

- The image shows a side-by-side comparison. On the left is the MINUS login page with a blue logo and the text 'Inicio de sesión'. It features input fields for 'Usuario' (containing 'gvera') and 'Contraseña' (masked with dots), and a green 'Iniciar sesión' button. A red arrow points to the 'Usuario' field. On the right is the Chrome DevTools Network tab. The 'Network' tab is selected, showing a list of requests. The first request, 'collect?v=2&tid=G-N5QSTR1...', is highlighted with a red arrow. The table columns are Name, Status, Type, Initiator, Size, Time, and Waterfall.

Name	Status	Type	Initiator	Size	Time	Waterfall
collect?v=2&tid=G-N5QSTR1...	204	ping	js?id=G-N5OST...	17 B	81 ms	

-

- ```
$ curl -s -X POST -H 'Content-Type: application/json' -d '{"username":"gvera","password":"AQUI-VA-EL-PASSWORD-EN-BASE64"}' https://eminusapi.uv.mx/eminusapi/api/auth
```

```
pato@equipoA~$ curl -s -X POST -H 'Content-Type: application/json' --data '{"username":"gvera","password":"Lq0wwB/PllJpk/OjSdmdQqEopFhUn/ScOPsVw+2AC4G6R/LvuEMqZKQtOxzmSz"}' https://eminusapi.uv.m
eminusapi/api/auth
{
 "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmYyZWlhaWQiOiJndmVyYSIsImpoOiAiSi6IMFiNWIIiDNDdl
mZmZtNQ04NS04YzgwLTtyZmZyTRMmFmZiIsInJvbEFkbWluIjoiaDI1ISIiImh0dHA6Ly9yZ2hlbWFzfENhtBHNVYAub3JnL
zLzlwMDUvMDUvaWRlbnpkdHkvY2hwaWZlZC1pNzc1NTt1MzYyIiwibmJmIjoxNDA4MDAwMTU1ODk1eT09IiwiaGF0eSI6IjE3MDcwMTc1N
SImlzcyGI6ImVtaW51cy51di5teCiSI6ImVtaW51cy51di5teCj9.eyJErOCFlFYWiiUEfKnmtQuXnlLt4Y6pf_I7O-X
M","tokenExpiration":{"2024-02-15T11:19:19.1404895100:00"},"tokenExpirationEpoch":1708017559,"esEs
```

12. Observe que estamos enviando una petición POST con un par de datos, nuestro nombre de usuario y contraseña de la UV. El servidor nos regresa una cadena donde viene nuestro token de acceso para poder realizar peticiones a la API de Eminus. Copie esta cadena llama token.
13. Ya con el token de identificación, vamos a volver a intentar obtener los datos de un usuario. Utilice la misma petición anterior, solo agregue un encabezado llamado **Autorization**.

```
$ curl -X GET -H "Authorization: Bearer TOKENCOPIADO"
https://eminusapi.uv.mx/eminusapi/api/Usuario/getUsuario

{"idUsuario":"gvera","nombre":"GUILLERMO
HUMBERTO","paterno":"VERA","materno":"AMARO","correo":"gvera@uv.mx","correoAlterno":null,"perteneceU
V":1,"esEstudiante":0,"tieneFotografia":1,"idCampus":1,"campus":"Xalapa","idDependencia":null,"idiom
a":1,"tema":1,"archivo":null,"ruta":"\\profiles\\vIo345GRJeyJ3k6WRMDtQ.png"}p
```

14. Observe como se regresan los valores de su usuario.
15. Para no estar escribiendo la cadena de token en cada petición, puede almacenarla en una variable.

```
$ TOKEN="eyJhbGciOiJIUzcy51di5teCJ9.L8mXbsODC12DGjuJlZBeefjEXEVsBaBxELz6eRQI314"
```

16. Y puede realizar las llamadas simplemente referenciando esta variable.

```
~$ curl -X GET -H "Authorization: Bearer ${TOKEN}"
https://eminusapi.uv.mx/eminusapi/api/Usuario/getUsuario
```

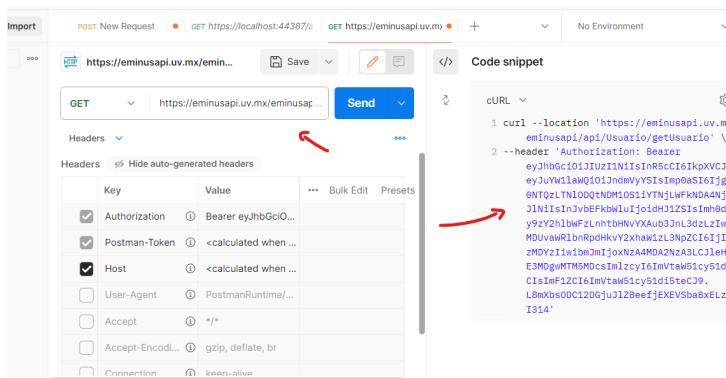
17. Pruebe con otro método de Eminus. Por ejemplo, el que obtiene todos los cursos disponibles.

```
pato@equipoA:~$ curl -X GET -H "Authorization: Bearer ${TOKEN}"
https://eminusapi.uv.mx/eminusapi/api/Cursos/getAllCourses
```

18. O el que obtiene los detalles de un curso en específico.

```
$ curl -X GET -H "Authorization: Bearer ${TOKEN}"
https://eminusapi.uv.mx/eminusapi/api/Cursos/obtieneCurso/63701
```

19. Cuando utiliza alguna herramienta gráfica de peticiones cliente hacia APIs como **Postman**, por debajo, está realizando estas peticiones usando Client URL.



20. Enhorabuena, ha concluido la práctica de métodos HTTP usando cURL